

1 BottomBar

BottomBar (представляет собой контейнер, который содержит в себе различные элементы)

1. **Контейнера** - сам View-компонент, содержащий элементы
2. **Элементов меню (Menu Items):**
 - Иконка
 - Текст подписи
 - Возможный индикатор уведомлений (badge)
3. **Фона** - может быть цветным или с эффектами (например, полупрозрачным)

```
xml
Copy
<com.google.android.material.bottomnavigation.BottomNavigationView
    android:id="@+id/bottom_navigation"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_gravity="bottom"
    app:menu="@menu/bottom_nav_menu"
    app:itemIconTint="@color/bottom_nav_colors"
    app:itemTextColor="@color/bottom_nav_colors"
    app:labelVisibilityMode="labeled"
    app:background="@color/white" />
```

Run HTML

```
xml
Copy
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item
        android:id="@+id/nav_home"
        android:icon="@drawable/ic_home"
        android:title="@string/home" />
    <item
        android:id="@+id/nav_search"
        android:icon="@drawable/ic_search"
        android:title="@string/search" />
    <item
        android:id="@+id/nav_profile"
        android:icon="@drawable/ic_profile"
        android:title="@string/profile" />
</menu>
```

```
java                                                                    Copy
BottomNavigationView bottomNav = findViewById(R.id.bottom_navigation);
bottomNav.setOnNavigationItemSelectedListener(item -> {
    switch (item.getItemId()) {
        case R.id.nav_home:
            // Обработка нажатия на Home
            return true;
        case R.id.nav_search:
            // Обработка нажатия на Search
            return true;
        case R.id.nav_profile:
            // Обработка нажатия на Profile
            return true;
    }
    return false;
});
```

Современные альтернативы

1. **Navigation Component с BottomNavigationView** - интеграция с Jetpack Navigation
2. **Custom View** - для сложных дизайнов
3. **MotionLayout** - для анимированных переходов

CoordinatorLayout – это контейнер, который расширяет FrameLayout.

Основные особенности CoordinatorLayout

1. **Координация между виджетами** – автоматическая синхронизация анимаций и событий (например, скрытие Toolbar при скроллинге).
2. **Поведения (Behaviors)** – можно задавать кастомные реакции на действия пользователя.
3. **Гибкость** – поддерживает сложные макеты с плавающими кнопками, скрывающимися панелями и т.д.
4. **Работа с Material Design** – идеально сочетается с AppBarLayout, CollapsingToolbarLayout, FloatingActionButton.

1. Добавление зависимости (build.gradle)

```
gradle                                                                    Copy
implementation 'com.google.android.material:material:1.6.0' // или актуальная версия
```

2. Пример разметки (XML)

```
xml                                                                    Copy
<?xml version="1.0" encoding="utf-8"?>
<androidx.coordinatorlayout.widget.CoordinatorLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <!-- AppBarLayout c CollapsingToolbarLayout -->
    <com.google.android.material.appbar.AppBarLayout
        android:layout_width="match_parent"
        android:layout_height="250dp">

        <com.google.android.material.appbar.CollapsingToolbarLayout
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            app:layout_scrollFlags="scroll|exitUntilCollapsed">

            <ImageView
                android:layout_width="match_parent"
                android:layout_height="match_parent"
                android:src="@drawable/header_image"
                app:layout_collapseMode="parallax"/>

            <androidx.appcompat.widget.Toolbar
                android:layout_width="match_parent"
                android:layout_height="?attr/actionBarSize"
                app:layout_collapseMode="pin"/>
        </com.google.android.material.appbar.CollapsingToolbarLayout>
    </com.google.android.material.appbar.AppBarLayout>
```

4-й пункт особенностей CoordinatorLayout: "Работа с Material Design"

CoordinatorLayout идеально интегрируется с компонентами **Material Design**, такими как:

- **AppBarLayout** – панель вверху экрана (например, **Toolbar**), которая может скрываться при скролле.
- **CollapsingToolbarLayout** – расширяемый/схлопывающийся заголовок.
- **FloatingActionButton (FAB)** – плавающая кнопка, которая может автоматически прятаться при прокрутке.
- **BottomSheet** – выезжающая снизу панель.

Примеры взаимодействия:

- При скролле **RecyclerView** **Toolbar** скрывается, а **FAB** плавно исчезает.
- **BottomNavigationView** остается фиксированным внизу, пока контент прокручивается.

Как связать BottomNavigationView с CoordinatorLayout

1. Простая фиксация BottomNavigationView внизу

Если нужно, чтобы `BottomNavigationView` всегда оставался внизу, а контент скроллился над ним:

```
xml Copy

<androidx.coordinatorlayout.widget.CoordinatorLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <!-- Основной контент (прокручиваемый) -->
    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/recyclerView"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:paddingBottom="56dp" <!-- Отступ для BottomNavigationView -->
        app:layout_behavior="@string/appbar_scrolling_view_behavior" />

    <!-- Фиксированный BottomNavigationView -->
    <com.google.android.material.bottomnavigation.BottomNavigationView
        android:id="@+id/bottomNavigationView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_gravity="bottom"
        app:menu="@menu/bottom_nav_menu" />

</androidx.coordinatorlayout.widget.CoordinatorLayout>
```

Осуществляет перемещение между экранами

Для создания новой папки нажимаем правой кнопкой мыши на "res" → "New" → "Android Resource Directory" (рисунок 1.1.3). Вводим название папки, выбираем тип ресурса из выпадающего списка (рисунок 1.1.4).

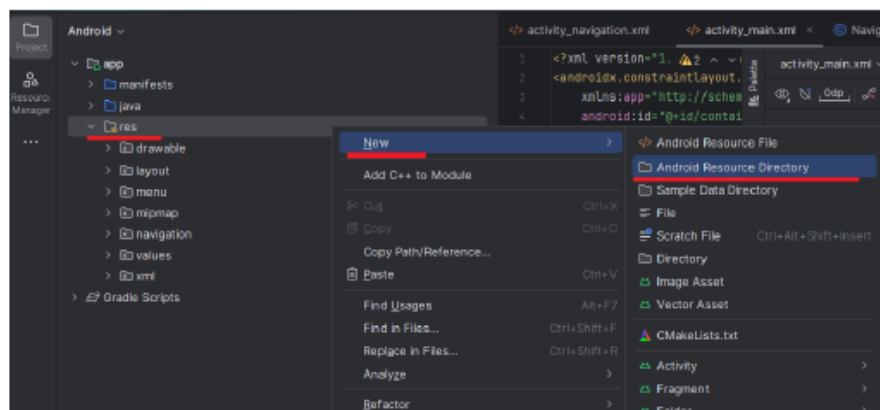


Рисунок 1.1.3 – Создание новой папки в проекте

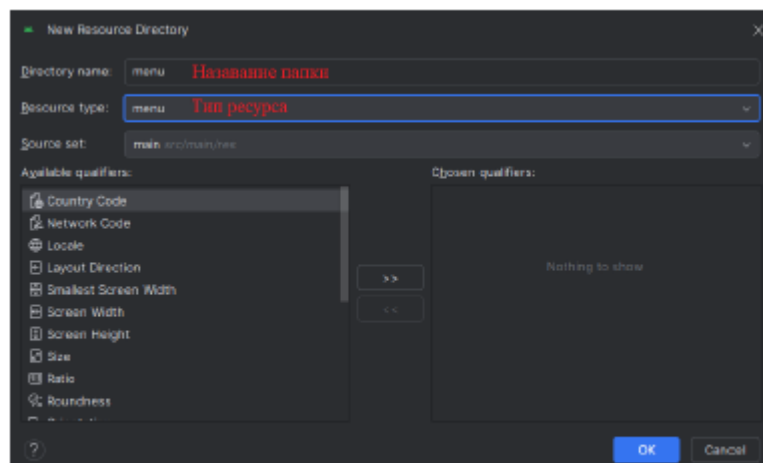


Рисунок 1.1.4 – Настройка создаваемой папки

Тег `<menu>` является корневым узлом файла и определяет меню, состоящее из одного или нескольких элементов `<item>` и `<group>`.

Элемент `<item>` представляет объект `MenuItem`, которой является одним из элементов меню. Этот элемент может содержать внутренний подэлемент `<menu>`, с помощью которого создается подменю.
подэлемент `<menu>`, с помощью которого создается подменю.

Элемент `<item>` включает следующие атрибуты, которые определяют его внешний вид и поведение:

- `android:id`: уникальный id элемента меню, который позволяет его опознать при выборе пользователем и найти через поиск ресурса по id;
- `android:icon`: ссылка на ресурс `drawable`, который задает изображение для элемента (`android:icon="@drawable/ic_help"`);
- `android:title`: ссылка на ресурс строки, содержащий заголовок элемента. По умолчанию имеет значение "Settings";

создать значок для отображения каждого из этих элементов. Чтобы создать значок, нажмите на "drawable" → "New" → "Image Asset" (рисунок 1.1.6).

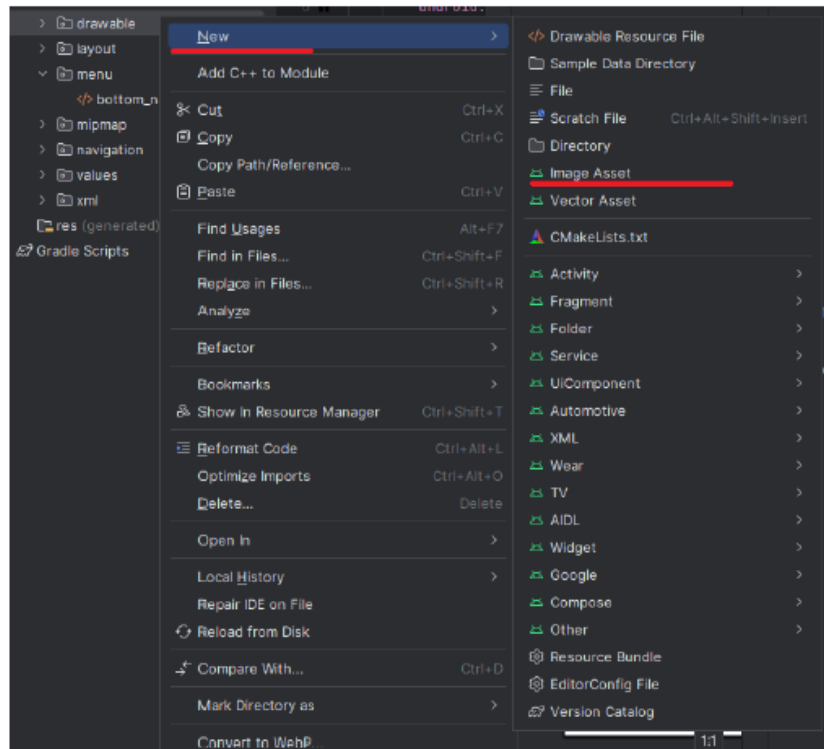


Рисунок 1.1.6 – Процесс создания новой иконки

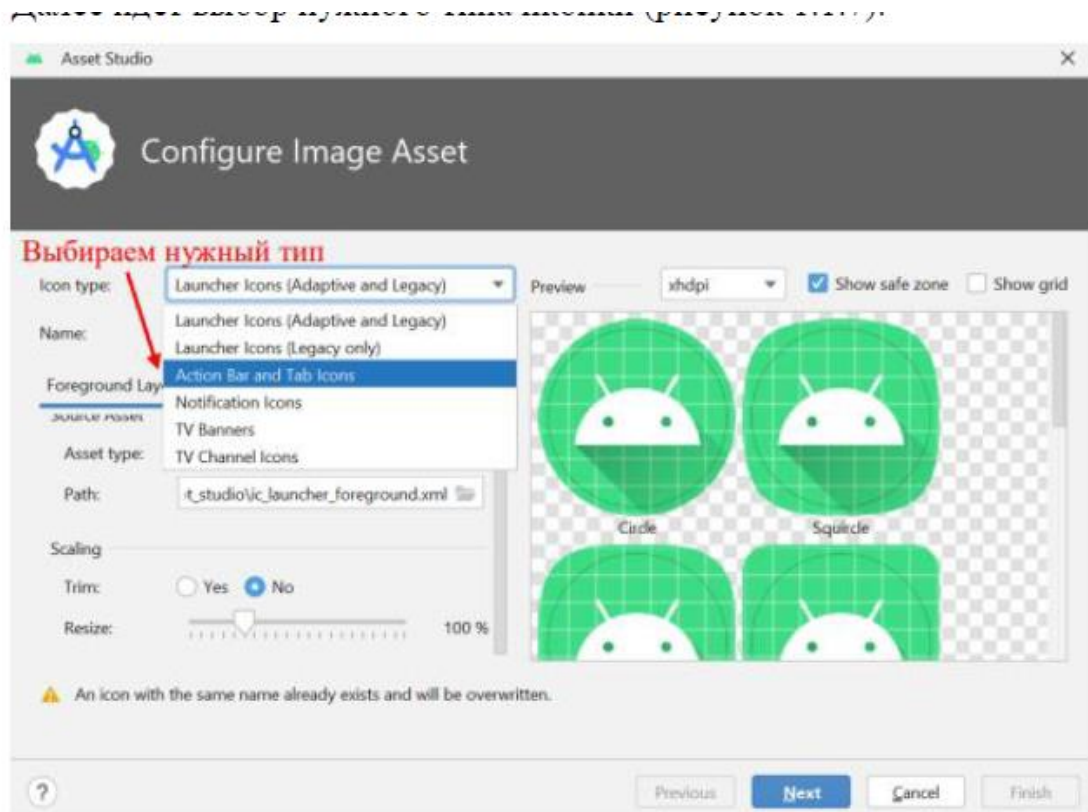


Рисунок 1.1.7 – Выбор необходимого типа иконки



В Android Studio при создании новой иконки (через **File → New → Image Asset**) предлагается несколько типов иконок, каждый из которых имеет свои особенности и назначение. Вот основные отличия:

1. Launcher Icons (Адаптивные и старые иконки приложения)

Adaptive Icon (API 26+, Android 8.0 и выше)

- **Состоит из двух слоёв:**
 - **Foreground** (передний план) – основная фигура (например, логотип).
 - **Background** (фон) – цвет или градиент.
- **Поддерживает анимации и различные формы** (круг, квадрат, сквош-иконка и т. д.), которые зависят от лаунчера пользователя.
- **Размеры:**
 - 108×108 dp (foreground), 432×432 dp (background).
- **Файлы:** Генерируются в `mirmap-anydpi-v26/`.

Legacy Icon (API 25 и ниже)

- **Одиночное изображение** без разделения на слои.
 - **Используется для старых версий Android.**
 - **Размеры:**
 - 48×48 dp (mdpi), 72×72 dp (hdpi) и т. д.
 - **Файлы:** Генерируются в `mirmap-*/`.
-

2. Action Bar / Tab Icons (Иконки для панели действий)

- **Используются в `Toolbar` или `TabLayout`.**
 - **Требуют прозрачного фона.**
 - **Размеры:**
 - 24×24 dp (для action bar), 32×32 dp (для tabs).
 - **Цвет:** Могут быть монохромными (настраиваются через `android:tint`).
-



3. Notification Icons (Иконки уведомлений)

- Должны быть полностью белыми с прозрачным фоном (система окрашивает их в белый цвет).
- Размеры: 24×24 dp (mdpi), 36×36 dp (hdpi) и т. д.
- Важно: Если иконка не монохромная, уведомление может отображаться некорректно.

4. TV Icons (Иконки для Android TV)

- Большой размер: 48×48 dp (mdpi), 72×72 dp (hdpi).
- Должны быть четкими из-за больших экранов.

5. Round Icons (Круглые иконки)

- Альтернатива для устройств с круглыми экранами (например, часы).
- Обрезаются системой до круга, поэтому важное содержимое должно быть в центре.

Сравнительная таблица

Тип иконки	Назначение	Размер (dp)	Цвета	Где используется
Adaptive Icon	Иконка приложения (API 26+)	108×108 (fg)	Любые	Лаунчер
Legacy Icon	Иконка приложения (старые ОС)	48×48 (mdpi)	Любые	Лаунчер (API ≤ 25)
Action Bar Icon	Кнопки в Toolbar	24×24	Монохром	Панель действий
Notification Icon	Уведомления	24×24 (mdpi)	Белый	Notification
TV Icon	Android TV	48×48 (mdpi)	Любые	TV-приложения
Round Icon	Устройства с круглым экраном	Как launcher icon	Любые	Часы, круглые дисплеи

Why is it not possible to use uppercase in naming resources in android?

Вопрос Лучший ответ Ещё 1 ответ

1. not to allow uppercase letter in resources is just to follow some convention strictly
2. you can not use two different types of resources of a same name in the same resource folder. for example say you have two resource files in drawable-hdpi. they are a.png and a.xml. Here while compiling the compiler will show error. But if you place a.png in drawable-hdpi and a.xml in drawable-mdpi, then it will not show any error as these two resources will be used in two totally different device types.



Each file inside folder is translated into java field name inside `R.java` class.

2

`drawable\myicon.png` -> `R.drawable.myicon`



Hence the reason for not using special characters inside file names, as they can no be used in Java names.



Refer [this](#)

Image Asset поможет создать следующие типы значков:

- иконки лаунчера;
- значки панели действий и вкладок;
- значки уведомлений.

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    BottomNavigationView bottomNavigationView = findViewById(R.id.bottom_navigation);
    bottomNavigationView.setOnNavigationItemSelectedListener(
        new BottomNavigationView.OnNavigationItemSelectedListener() {
            @Override
            public boolean onNavigationItemSelected(@NonNull MenuItem item) {
                if (item.getItemId() == R.id.home){
                    // обработка выбора раздела "Домой"
                    Toast.makeText(context, MainActivity.this, text: "Домой", Toast.LENGTH_LONG).show();
                    return true;
                }
                else if (item.getItemId()==R.id.settings){
                    // обработка выбора раздела "Настройки"
                    Toast.makeText(context, MainActivity.this, text: "Настройки", Toast.LENGTH_LONG).show();
                }
                return false;
            }
        }
    );
}
```

Рисунок 1.1.12 – Реализация логики обработчика событий нижнего меню

Чтобы понять, какой пункт меню выбран, вначале получаем его идентификатор через `item.getItemId()`. Затем пробегаемся в конструкции `if...else` и выбираем нужный вариант и в зависимости от выбора производим определенные действия (рисунки 1.1.13-1.1.14).

2 ActionBar

ActionBar уже встроен в стандартные темы Activity начиная с API Level 11 (Honeycomb). Для его использования необходимо убедиться, что выбранная Activity наследуется от AppCompatActivity, и используется одну из тем Theme.AppCompat. ActionBar будет отображаться вверху страницы (рисунок 1.2.1).



ActionBar в Android Studio (Java версия)

ActionBar — это верхняя панель приложения, которая отображает название, навигационные элементы и меню. Вот как работать с ним в Java.

1. Минимально необходимые атрибуты

Убедитесь, что в `styles.xml` используется тема с `ActionBar`:

```
xml                                                                    Copy
<style name="AppTheme" parent="Theme.AppCompat.Light.DarkActionBar">
  <item name="colorPrimary">@color/colorPrimary</item>
  <item name="colorPrimaryDark">@color/colorPrimaryDark</item>
</style>

Run HTML
```

2. Создание ActionBar в коде (Java)

Активация ActionBar в Activity

```
java                                                                    Copy
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        // Включение ActionBar (если он скрыт)
        if (getSupportActionBar() != null) {
            getSupportActionBar().show();
        }

        // Настройка заголовка
        getSupportActionBar().setTitle("Моё приложение");
        // Альтернативный способ
        getSupportActionBar().setDisplayShowTitleEnabled(true);
    }
}
```



В **Android Studio** **Toolbar** используется чаще, чем **ActionBar**, по нескольким ключевым причинам:

1. Гибкость и кастомизация

- **Toolbar** — это просто **View** (наследник **ViewGroup**), поэтому его можно:
 - Размещать в любом месте макета (не только вверху экрана).
 - Легко стилизовать (цвета, иконки, отступы).
 - Добавлять произвольные элементы (кнопки, поиск, меню).
- **ActionBar** жестко привязан к верху экрана и сложнее изменяется.

2. Совместимость и Material Design

- **Toolbar** — часть библиотеки **AndroidX AppCompat**, что обеспечивает:
 - Единый стиль на всех версиях Android (начиная с API 14).
 - Поддержку **Material Design** (через **Theme.MaterialComponents**).
- **ActionBar** устарел и не поддерживает многие современные фишки.

3. Управление через XML и код

- **Toolbar** можно настраивать как в **layout-файле**, так и программно:

xml

Copy

```
<androidx.appcompat.widget.Toolbar
    android:id="@+id/toolbar"
    android:layout_width="match_parent"
    android:layout_height="?attr/actionBarSize"
    android:background="?attr/colorPrimary" />
```

▶ Run HTML

kotlin

Copy

```
setSupportActionBar(toolbar) // Делаем Toolbar заменой ActionBar
supportActionBar?.title = "Мое приложение"
```

- **ActionBar** управляется только через код и темы.



4. Способы добавления элементов

1. Через XML-меню (res/menu)

Стандартный способ (см. выше).

2. Программное добавление

```
java                                                                    Copy

@Override
public boolean onCreateOptionsMenu(Menu menu) {
    menu.add(0, 100, 0, "Динамическая кнопка").setIcon(R.drawable.ic_add);
    return true;
}
```

3. Кастомные View в ActionBar

```
java                                                                    Copy

if (getSupportActionBar() != null) {
    getSupportActionBar().setDisplayShowCustomEnabled(true);
    getSupportActionBar().setCustomView(getLayoutInflater().inflate(R.layout.custom_action_bar,
    null));
}
```

Однако такой ActionBar не несет никакой пользы. Для дальнейшей работы его необходимо настроить. В активности можно настроить ActionBar на отображение какого-либо строкового значения (рисунок 1.2.2).

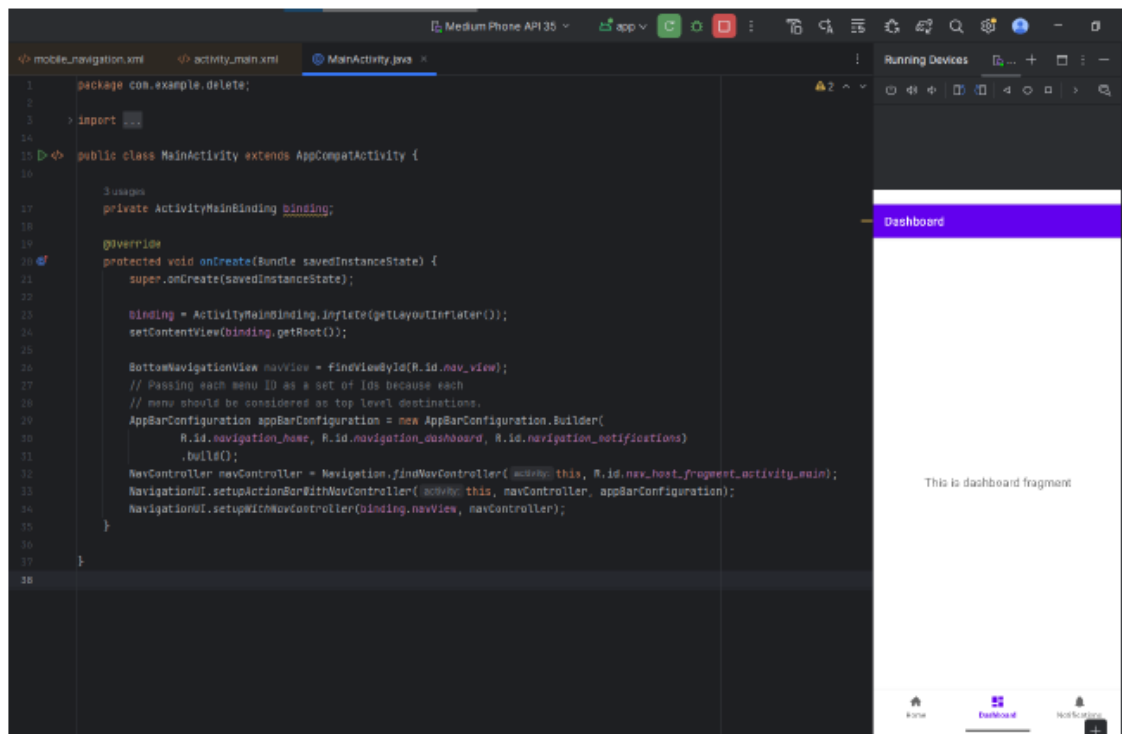


Рисунок 1.2.2 – Реализация логики обработки событий для ActionBar

Однако этого тоже может быть недостаточно, так как нам может понадобиться быстрой доступ к некоторым элементам.

Используем ранее созданный файл `bottom_nav_menu.xml`, только немного его изменим, добавив элементам меню некоторые параметры для наглядности (рисунок 1.2.3).

- `android:orderInCategory`: значение этого атрибута определяет положение элемента в `ActionBar`. Есть два способа определить положение различных пунктов меню. Первый - предоставить одинаковое значение этого атрибута для всех элементов, и позиция будет определена в том же порядке, в каком они объявлены в коде. Второй способ - предоставить разные числовые значения для всех элементов, и тогда элементы будут располагаться в соответствии с порядком возрастания значения этого атрибута;
- `app:showAsAction`: этот атрибут определяет, как элемент будет присутствовать на панели действий.

На выбор для значения поля `app:showAsAction` предлагается четыре возможных флага:

- `always`: постоянно отображать элемент на панели действий;
- `ifRoom`: сохранить элемент, если есть свободное место;
- `never`: с этим флагом элемент не будет отображаться в виде значка в `ActionBar`, но будет присутствовать в меню переполнения;
- `withText`: чтобы представить элемент одновременно в виде значка и заголовка, можно дополнить этот флаг флагом `always` или `ifRoom` (`always|withText` or `ifRoom|withText`).

(меню)

описание. Чтобы вывести его на экран, надо использовать его в классе `Activity`. Для этого надо переопределить метод `onCreateOptionsMenu`.

Метод `getMenuInflater` возвращает объект `MenuInflater`, у которого вызывается метод `inflate()`. Этот метод в качестве первого параметра принимает ресурс, представляющий наше описание меню в `xml`, и наполняет им объект `menu`, переданный в качестве второго параметра.

Теперь, если мы нажмем на любой из пунктов меню, то ничего не произойдет. Чтобы привязать к меню действия, нам надо переопределить в классе `activity` метод `onOptionsItemSelected` (рисунок 1.2.4).

```
<> bottom_nav_menu.xml <> activity_navigation.xml <> activity_main.xml MainActivity.java x AndroidManifest.xml NavigationA
19 public class MainActivity extends AppCompatActivity {
24     protected void onCreate(Bundle savedInstanceState) {
27         binding = ActivityMainBinding.inflate(getLayoutInflater());
28         setContentView(binding.getRoot());
29
30         BottomNavigationView navView = findViewById(R.id.nav_view);
31         // Passing each menu ID as a set of Ids because each
32         // menu should be considered as top level destinations.
33         AppBarConfiguration appBarConfiguration = new AppBarConfiguration.Builder(
34             R.id.navigation_home, R.id.navigation_dashboard, R.id.navigation_notifications)
35             .build();
36         NavController navController = Navigation.findNavController( activity: this, R.id.nav_host_fragment_activity_main);
37         NavigationUI.setupActionBarWithNavController( activity: this, navController, appBarConfiguration);
38         NavigationUI.setupWithNavController(binding.navView, navController);
39     }
40
41     @Override
42     public boolean onCreateOptionsMenu( Menu menu ) {
43         getMenuInflater().inflate(R.menu.bottom_nav_menu, menu);
44         return super.onCreateOptionsMenu(menu);
45     }
46
47     @Override
48     public boolean onOptionsItemSelected( @NonNull MenuItem item ) {
49         //Определение нажатого элемента
50         if (item.getItemId() == R.id.navigation_home) {
51             // обработка выбора раздела "Домой"
52             Toast.makeText( context: MainActivity.this, text: "Домой", Toast.LENGTH_LONG).show();
53             return true;
54         } else if (item.getItemId() == R.id.navigation_notifications) {
55             // обработка выбора раздела "Настройки"
56             Toast.makeText( context: MainActivity.this, text: "Настройки", Toast.LENGTH_LONG).show();
57         }
58         return super.onOptionsItemSelected(item);
59     }
60 }
```

Toast – короткое сообщение

kotlin

Copy

```
Toast.makeText(context, "Сообщение", Toast.LENGTH_SHORT).show()
```

- `context` – контекст активности или приложения (например, `this` внутри `Activity`).
- `"Сообщение"` – текст, который нужно показать.
- `Toast.LENGTH_SHORT` – длительность отображения.

1. Что означает `@NonNull` ?

- Если параметр, поле или метод помечен `@NonNull`, это означает, что:
 - Передача `null` вызовет предупреждение (в Android Studio) или ошибку компиляции (если включена строгая проверка).
 - Разработчик обязан гарантировать, что значение не `null`.
- Используется для предотвращения `NullPointerException` (NPE).

NonNull

- В Android Studio и Kotlin/Java аннотация `@NonNull` указывает, что переменная, параметр метода или возвращаемое значение **не могут быть** `null`. Это часть поддержки **аннотаций для проверки null-безопасности** в Android (чаще всего из пакета `androidx.annotation` или `org.jetbrains.annotations`).

3 NavigatonDrawer

`NavigationDrawer` (выдвижное меню) - это популярный паттерн навигации в Android-приложениях. Вот как его реализовать с минимальным количеством атрибутов и кода.

1. Основные компоненты

Для работы `NavigationDrawer` необходимы:

- `DrawerLayout` - корневой контейнер
- Основной контент (например, `FrameLayout`)
- `NavigationView` - само выдвижное меню


```
activity_main.xml MainActivity.java activity_drawer.xml x
<?xml version="1.0" encoding="utf-8"?>
<androidx.drawerlayout.widget.DrawerLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/drawer_layout"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
    <!-- Основной контент -->
    <FrameLayout
        android:id="@+id/content_frame"
        android:layout_width="match_parent"
        android:layout_height="match_parent">
        <!-- Добавьте сюда свой контент -->
    </FrameLayout>
    <!-- Выдвижная панель -->
    <com.google.android.material.navigation.NavigationView
        android:id="@+id/nav_view"
        android:layout_width="wrap_content"
        android:layout_height="match_parent"
        android:layout_gravity="start"
        app:menu="@menu/bottom_nav_menu" />
</androidx.drawerlayout.widget.DrawerLayout>
```

getSupportActionBar

`getSupportActionBar()` is the same as `getActionBar()` , but comes from the Android Support library.

Android Studio is recommending you switch to `getSupportActionBar()` because, if you did not, your application would not be backwards-compatible with previous versions of Android.

Android Support Library (библиотека поддержки Android) — это набор совместимых библиотек, предоставляемых Google для обеспечения обратной совместимости новых функций Android на старых версиях ОС, а также для добавления дополнительных компонентов и возможностей, которых нет в стандартном API.

onOptionsItemSelected

Когда вы переопределяете метод `onOptionsItemSelected()` в Android и вызываете `super.onOptionsItemSelected(item)`, это делается для **корректной обработки системных пунктов меню** (например, кнопки "Назад" в `ActionBar` или стандартных действий). Разберём подробнее:

◆ Зачем вызывать `super.onOptionsItemSelected()` ?

1. Обработка стандартных элементов меню

Некоторые системные пункты меню (например, кнопка "Home" в `ActionBar` или навигационные действия) обрабатываются родительским классом (`AppCompatActivity`, `Activity`). Если не вызвать `super`, они **перестанут работать**.

2. Следование контракту `@Override`

В документации Android явно указано, что при переопределении этого метода нужно вызывать родительскую реализацию, если обработчик не обработал пункт меню:

```
java                                                                    Copy
@Override
public boolean onOptionsItemSelected(MenuItem item) {
    if (item.getItemId() == R.id.my_item) {
        // Ваша логика
        return true; // Пункт меню обработан
    }
    return super.onOptionsItemSelected(item); // Передаём остальное родителю
}
```

3. Избежание "тихих" багов

Если не вызвать `super`, могут возникнуть проблемы:

- Не сработает кнопка "Назад" в `ActionBar`.
- Не будут работать навигационные действия (если используется `NavController`).

ActionBarDrawerToggle

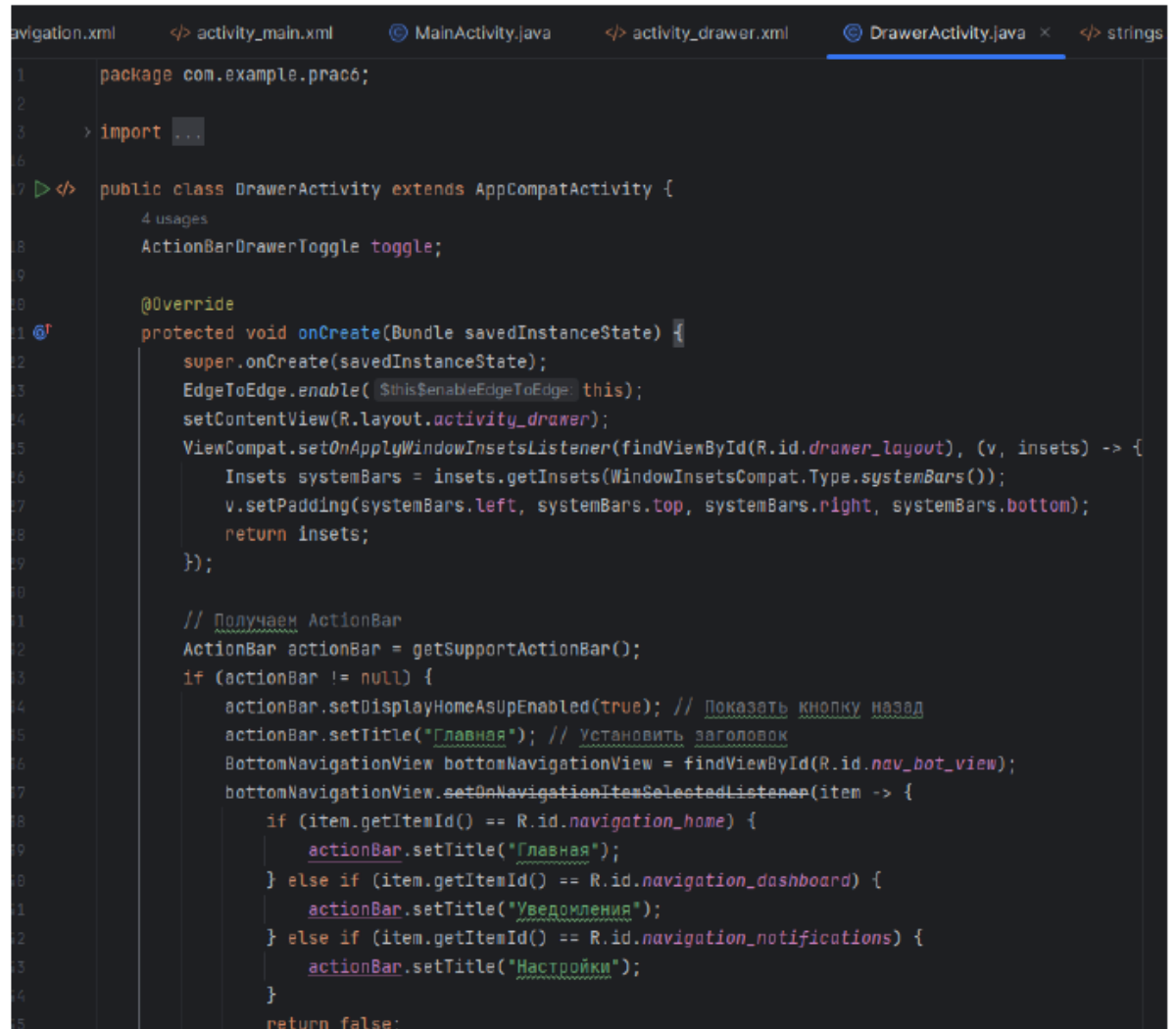
Андрей Ковалев [@kvalov](#) [@kvalov](#) [@kvalov](#) [@kvalov](#) [@kvalov](#) [@kvalov](#) [@kvalov](#) [@kvalov](#) [@kvalov](#) [@kvalov](#)

Чтобы использовать ActionBarDrawerToggle, нужно создать экземпляр при помощи конструктора со следующими аргументами:

- Activity, в котором размещается боковая панель навигации;
- DrawerLayout – drawable ресурс, используемый в качестве индикатора панели. Стандартный навигационный значок панели доступен в [Download the Action Bar Icon Pack](#);
- Строковый ресурс для обозначения открытой панели (для специальных возможностей);
- Строковый ресурс для обозначения закрытой панели (для специальных возможностей).

Drawer

Кроме этого, ActionBar можно использовать вместе с BottomNavigationView для создания единой системы навигации. В этом случае ActionBar обычно служит для отображения контекстной информации (например, заголовка текущей страницы), а BottomNavigationView для навигации между основными разделами приложения (рисунки 1.3.6-1.3.7).



```
1 package com.example.prac6;
2
3 > import ...
4
5
6
7 public class DrawerActivity extends AppCompatActivity {
8     ActionBarDrawerToggle toggle;
9
10
11     @Override
12     protected void onCreate(Bundle savedInstanceState) {
13         super.onCreate(savedInstanceState);
14         EdgeToEdge.enable(this);
15         setContentView(R.layout.activity_drawer);
16         ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.drawer_layout), (v, insets) -> {
17             Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
18             v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
19             return insets;
20         });
21
22         // Получаем ActionBar
23         ActionBar actionBar = getSupportActionBar();
24         if (actionBar != null) {
25             actionBar.setDisplayHomeAsUpEnabled(true); // Показывать кнопку назад
26             actionBar.setTitle("Главная"); // Установить заголовок
27             BottomNavigationView bottomNavigationView = findViewById(R.id.nav_bot_view);
28             bottomNavigationView.setOnNavigationItemSelectedListener(item -> {
29                 if (item.getItemId() == R.id.navigation_home) {
30                     actionBar.setTitle("Главная");
31                 } else if (item.getItemId() == R.id.navigation_dashboard) {
32                     actionBar.setTitle("Уведомления");
33                 } else if (item.getItemId() == R.id.navigation_notifications) {
34                     actionBar.setTitle("Настройки");
35                 }
36             });
37         }
38         return false;
39     }
40 }
```

```

_navigation.xml    </> activity_main.xml    MainActivity.java    </> activity_drawer.xml    DrawerActivity.java    x    </> strin
17    public class DrawerActivity extends AppCompatActivity {
21        protected void onCreate(Bundle savedInstanceState) {
27            bottomNavigationView.setOnNavigationItemSelectedListener(item -> {
40                } else if (item.getItemId() == R.id.navigation_dashboard) {
41                    actionBar.setTitle("Уведомления");
42                } else if (item.getItemId() == R.id.navigation_notifications) {
43                    actionBar.setTitle("Настройки");
44                }
45                return false;
46            });
47        }
48        DrawerLayout drawer = findViewById(R.id.drawer_layout);
49        toggle = new ActionBarDrawerToggle(
50            activity, DrawerActivity.this, drawer, R.string.drawer_open, R.string.drawer_close);
51        if (drawer != null) {
52            drawer.addDrawerListener(toggle);
53        }
54        toggle.syncState();
55        // to make the Navigation drawer icon always appear on the action bar
56        getSupportActionBar().setDisplayHomeAsUpEnabled(true);
57    }
58    // Обработка нажатия на иконку меню в ActionBar для открытия и закрытия Drawer
59    @Override
60    public boolean onOptionsItemSelected(MenuItem item) {
61        if (toggle.onOptionsItemSelected(item)) {
62            return true;
63        }
64        return super.onOptionsItemSelected(item);
65    }
66 }

```